



Northeastern University
College of Engineering, Toronto
DAMG 7370 — Data Engineering



Airline Delay Analysis

Cloud-Based Data Engineering Pipeline on Microsoft Azure

Final Project Report — April 2026

71.5M Raw Records	63,986 Gold Records	20 SQL Queries	5 Azure Services
-----------------------------	-------------------------------	--------------------------	----------------------------

TEAM MEMBERS	
Imran Khan Pathan <i>pathan.i@northeastern.edu</i>	Robins Ranjan <i>ranjan.r@northeastern.edu</i>
Durga Dhana Sree Chillukuri <i>chillukuri.d@northeastern.edu</i>	Adithya Anand <i>anand.ad@northeastern.edu</i>
Course: DAMG 7370 Northeastern University	

Table of Contents

- 1. Executive Summary 1
- 2. Introduction 2
 - 2.1 Background and Motivation 2
 - 2.2 Project Objectives 2
 - 2.3 Business Questions 2
 - 2.4 Scope and Limitations 2
- 3. Dataset Description 3
 - 3.1 Data Source 3
 - 3.2 Dataset Statistics 3
 - 3.3 Raw Dataset Schema — 28 Columns 3
 - 3.4 Airline Carriers in the Dataset 3
- 4. System Architecture 4
 - 4.1 Medallion Architecture Overview 4
 - 4.2 Layer Descriptions 4
 - 4.3 Azure Infrastructure 4
- 5. Pipeline Implementation 5
 - 5.1 Data Collection — Bronze Layer 5
 - 5.1.1 Source Dataset 5
 - 5.1.2 Upload Process 5
 - 5.2 Data Transformation — Silver Layer 5
 - 5.2.1 Databricks Cluster Configuration 5
 - 5.2.2 Transformation Logic 5
 - 5.3 Data Aggregation — Gold Layer 5
 - 5.3.1 Aggregation Design 5
 - 5.4 Azure Data Factory — Orchestration 5
- 6. Synapse Analytics — SQL Query Layer 6
 - 6.1 Workspace Configuration 6
 - 6.2 OPENROWSET Query Pattern 6
 - 6.3 The 20 Analytical Queries 6
 - 6.4 Complete SQL Query Listing 6

Table of Contents

7. Power BI Dashboard 7

7.1 Data Connection 7

7.2 Power Query Fix — Metadata File Error..... 7

7.3 Data Model 7

7.4 Dashboard Structure 7

8. Challenges and Solutions 8

9. Key Findings and Analytical Insights..... 9

9.1 Airline Delay Performance..... 9

9.2 Seasonal Delay Patterns..... 9

9.3 Year-Over-Year Trends..... 9

9.4 Cancellation Analysis 9

9.5 Distance and Delay Relationship..... 9

10. Conclusion 10

10.1 Future Work 10

11. Team Member Contributions..... 11

References..... 12

1. Executive Summary

This report presents a comprehensive account of the Airline Delay Analysis project developed for DAMG 7370 — Data Engineering at Northeastern University. The project delivers a production-grade, end-to-end cloud data engineering pipeline built entirely on Microsoft Azure, processing over 71 million U.S. domestic airline flight records spanning nine years (2009–2017).

The pipeline is architected using the industry-standard Medallion Architecture, progressing data through three progressive layers of refinement — Bronze (raw ingestion), Silver (cleaning and transformation), and Gold (aggregation and business metrics) — before delivering interactive analytical insights through a Microsoft Power BI dashboard.

The pipeline was designed to answer five core business questions about U.S. airline operational performance: identifying which carriers experience the highest delays, uncovering seasonal delay patterns, tracking year-over-year improvement trends, analysing cancellation causes, and understanding the relationship between flight distance and delay accumulation.

Metric	Value
Raw Records Processed	71,526,575
Gold Layer Aggregated Records	63,986
Azure Cloud Services Used	5 (ADLS Gen2, Databricks, ADF, Synapse, Power BI)
Synapse SQL Queries Written	20
Power BI Dashboard Pages	3
Dataset Time Span	2009 – 2017 (9 years)
Source Files	12 CSV files (~2 GB total)
Airline Carriers Covered	14 U.S. domestic carriers

Table 1: Project Summary Statistics

2. Introduction

2.1 Background and Motivation

Flight delays represent one of the most significant operational challenges in the U.S. commercial aviation industry. According to the Bureau of Transportation Statistics (BTS), flight delays cost airlines, passengers, and the broader economy billions of dollars annually in lost productivity, increased fuel burn, crew overtime, and passenger compensation.

Understanding the patterns, causes, and trends of airline delays is therefore of immense value to airline operators, airport authorities, regulators, and passengers alike. However, the sheer scale of flight data — hundreds of millions of records spanning years — makes traditional spreadsheet-based analysis impractical. This project addresses that gap by building a scalable cloud data pipeline that can handle the full volume of data and deliver insights in real time through an interactive dashboard.

2.2 Project Objectives

The primary objectives of this project were:

- Design and implement a complete end-to-end data engineering pipeline on Microsoft Azure
- Apply the Medallion Architecture pattern (Bronze → Silver → Gold) to progressively refine raw flight data
- Perform large-scale data transformation using Apache Spark (PySpark) on Azure Databricks
- Develop 20 analytical SQL queries using Azure Synapse Analytics to answer business questions
- Build an interactive Power BI dashboard to visualise key findings for business stakeholders
- Document all technical decisions, challenges, and solutions encountered during implementation

2.3 Business Questions

Five core business questions guided the analytical direction of this project:

- Which U.S. airlines have the highest average departure and arrival delays?
- Are flight delays improving or worsening on a year-over-year basis across the network?
- What seasonal and monthly patterns exist in delay trends?
- What is the cancellation rate by carrier, and how has it trended over time?
- Is there a relationship between flight distance and the accumulation of arrival delays?

2.4 Scope and Limitations

This project covers U.S. domestic flight records from 2009 to 2017 sourced from the Kaggle Airline Delay Analysis dataset, which is itself derived from the official U.S. Bureau of Transportation Statistics On-Time Performance database. The dataset covers 14 major and regional U.S. carriers across all domestic routes.

Key limitations of this project include:

- The dataset ends in 2017 and does not reflect post-2017 developments, including the COVID-19 pandemic disruptions to the aviation industry.
- International flights are not included; analysis is restricted to U.S. domestic routes only.
- Individual flight-level analysis (specific flight numbers, tail numbers) was out of scope for the gold aggregation layer.
- Delay cause breakdown metrics (carrier, weather, NAS, security, late aircraft) are available in the silver layer but were not carried forward into the gold aggregation — a noted area for future enhancement.
- Airport and route-level analysis is not possible at the gold layer due to the aggregation granularity chosen (carrier + date).

3. Dataset Description

3.1 Data Source

The dataset used in this project is the Airline Delay and Cancellation dataset published on Kaggle, sourced directly from the U.S. Bureau of Transportation Statistics (BTS) On-Time Performance database. The BTS is the official U.S. government agency responsible for collecting, analysing, and disseminating transportation data, and all U.S. airlines operating scheduled domestic service are legally required to report their on-time performance data to the BTS.

The dataset spans nine calendar years from 2009 to 2017 and is distributed across 12 separate CSV files totalling approximately 2 GB of raw data.

3.2 Dataset Statistics

Attribute	Value
Total Records	71,526,575 flight records
Number of Source Files	12 CSV files
Time Period	January 2009 to December 2017 (9 years)
Total Raw Size	Approximately 2 GB
Columns per Record	28 columns
Carriers Covered	14 U.S. airline carriers
Geographic Scope	All U.S. domestic routes
Data Authority	U.S. Bureau of Transportation Statistics (BTS)

Table 2: Dataset Overview Statistics

3.3 Raw Dataset Schema — 28 Columns

The raw dataset contains 28 columns across five logical categories:

#	Column Name	Category	Description
1	FL_DATE	Flight Identity	Flight date (YYYY-MM-DD)
2	OP_CARRIER	Flight Identity	Operating carrier IATA code
3	OP_CARRIER_FL_NUM	Flight Identity	Carrier flight number
4	ORIGIN	Route	Origin airport IATA code
5	DEST	Route	Destination airport IATA code
6	CRS_DEP_TIME	Timing	Scheduled departure time (HHMM)
7	DEP_TIME	Timing	Actual departure time (HHMM)

#	Column Name	Category	Description
8	DEP_DELAY	Delay	Departure delay in minutes
9	TAXI_OUT	Timing	Taxi-out time in minutes
10	WHEELS_OFF	Timing	Wheels-off time (HHMM)
11	WHEELS_ON	Timing	Wheels-on time at destination (HHMM)
12	TAXI_IN	Timing	Taxi-in time in minutes
13	CRS_ARR_TIME	Timing	Scheduled arrival time (HHMM)
14	ARR_TIME	Timing	Actual arrival time (HHMM)
15	ARR_DELAY	Delay	Arrival delay in minutes
16	CANCELLED	Status	Flight cancelled flag (0.0 / 1.0)
17	CANCELLATION_CODE	Status	Cancellation reason (A / B / C / D)
18	DIVERTED	Status	Flight diverted flag (0.0 / 1.0)
19	CRS_ELAPSED_TIME	Timing	Scheduled flight duration (minutes)
20	ACTUAL_ELAPSED_TIME	Timing	Actual flight duration (minutes)
21	AIR_TIME	Timing	Airborne time in minutes
22	DISTANCE	Route	Flight distance in miles
23	CARRIER_DELAY	Delay Cause	Delay caused by carrier (minutes)
24	WEATHER_DELAY	Delay Cause	Delay caused by weather (minutes)
25	NAS_DELAY	Delay Cause	Delay caused by National Air System (minutes)
26	SECURITY_DELAY	Delay Cause	Delay caused by security issues (minutes)
27	LATE_AIRCRAFT_DELAY	Delay Cause	Delay from late incoming aircraft (minutes)
28	Unnamed: 27	Artifact	Empty trailing column from CSV export

Table 3: Raw Dataset Column Schema (28 Columns)

3.4 Airline Carriers in the Dataset

Code	Airline Name	Type
AA	American Airlines	Major Legacy Carrier
AS	Alaska Airlines	Major Carrier
B6	JetBlue Airways	Low-Cost Carrier
DL	Delta Air Lines	Major Legacy Carrier
EV	ExpressJet Airlines	Regional Carrier
F9	Frontier Airlines	Ultra Low-Cost Carrier
HA	Hawaiian Airlines	Regional / Island Carrier
MQ	Envoy Air (American Eagle)	Regional Carrier
NK	Spirit Airlines	Ultra Low-Cost Carrier
OO	SkyWest Airlines	Regional Carrier
UA	United Airlines	Major Legacy Carrier
US	US Airways (merged into AA, 2015)	Major Legacy Carrier
VX	Virgin America (acquired by Alaska, 2016)	Low-Cost Carrier
WN	Southwest Airlines	Low-Cost Carrier

Table 4: U.S. Airline Carriers in the Dataset

4. System Architecture

4.1 Medallion Architecture Overview

This project implements the Medallion Architecture, an industry-standard data lake design pattern promoted by Databricks and widely adopted across enterprise cloud platforms including Microsoft Fabric, AWS Lake Formation, and Google Cloud Dataplex. The architecture organises data into three progressive layers, each adding increasing levels of quality, structure, and business value.

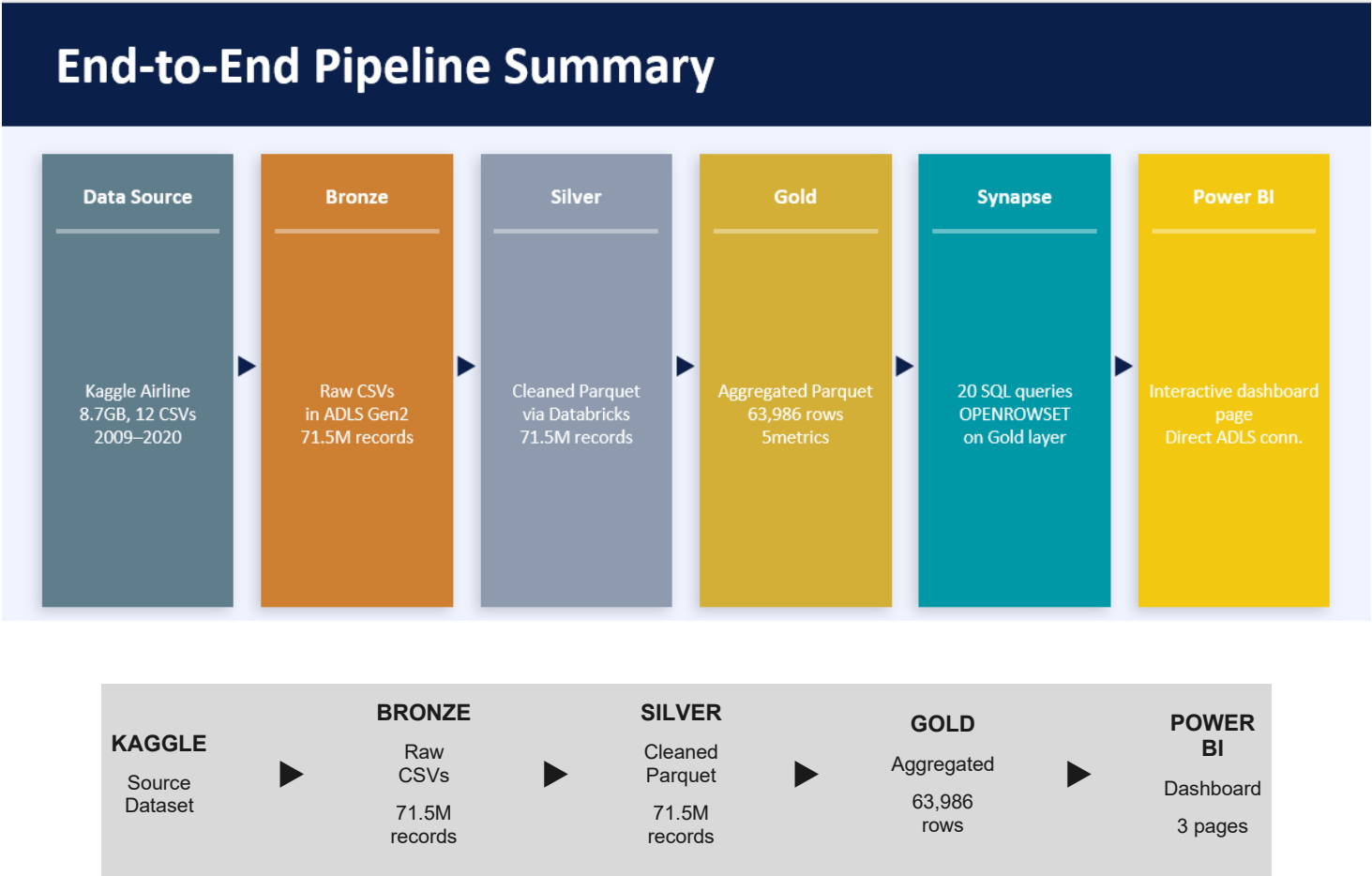


Figure 1: End-to-End Pipeline Architecture — Kaggle Source to Power BI Dashboard

4.2 Layer Descriptions

Layer	Container	Records	Purpose
Bronze	airlinedelaylake/bronze	71,526,575	Raw, unmodified CSV files as downloaded from Kaggle. No transformations. Serves as the immutable source of truth and enables full re-processing at any time.
Silver	airlinedelaylake/silver	71,526,575	Cleaned Parquet files with correct data types applied via PySpark. Zero rows dropped. Computationally queryable and schema-validated.
Gold	airlinedelaylake/gold	63,986	Aggregated Parquet files grouped by carrier and date. Contains 5 pre-computed business metrics. Optimised for Power BI dashboards.

Table 5: Medallion Architecture Layer Summary

4.3 Azure Infrastructure

All Azure resources were provisioned under a single Resource Group named airline-delay-project in the East US region. The exception is Synapse Analytics, deployed in West US 2 due to tenant-level SQL provisioning restrictions in the Northeastern University Azure subscription. All resources carry three mandatory governance tags.

Service	Resource Name	Region	Role
Azure Data Lake Gen2	airlinedelaylake	East US	Primary storage — bronze, silver, gold containers
Azure Databricks	airline-databricks	East US	PySpark transformation notebooks (airline-cluster)
Azure Data Factory	airline-adf-project	East US	Pipeline orchestration & scheduling
Azure Synapse Analytics	airline-synapse-project	West US 2	Serverless SQL querying on gold Parquet layer
Power BI Desktop	Local Application	N/A	Dashboard visualisation connected to ADLS Gen2

Table 6: Azure Infrastructure Resources

5. Pipeline Implementation

5.1 Data Collection — Bronze Layer

5.1.1 Source Dataset

The Kaggle Airline Delay Analysis dataset was downloaded as 12 separate CSV files. The files are named sequentially (471409_0.csv through 471409_11.csv) and cover different subsets of the 2009–2017 time period. The 2013 calendar year is split across two files due to its volume.

File	Year(s)	Approx. Size
471409_0.csv	2009	~140 MB
471409_1.csv	2010	~150 MB
471409_2.csv	2011	~160 MB
471409_3.csv	2012	~170 MB
471409_4.csv	2013 (Part 1)	~180 MB
471409_5.csv	2013 (Part 2)	~100 MB
471409_6.csv	2014	~180 MB
471409_7.csv	2015	~185 MB
471409_8.csv	2016	~190 MB
471409_9.csv	2017	~160 MB
471409_10.csv	2017 (supplement)	~80 MB
471409_11.csv	Multi-year supplement	~50 MB

Table 7: Source CSV Files in Bronze Layer

5.1.2 Upload Process

GitHub's 25 MB per-file size limit made version-control-based transfer infeasible for files ranging from 100 MB to over 300 MB each. The files were therefore uploaded directly to the Azure Data Lake bronze container using the Azure Portal's native blob upload interface — the correct industry practice for large dataset ingestion into cloud storage.

After upload, verification was performed in Azure Databricks by reading all files into a single PySpark DataFrame:

```
# Read all CSV files from the bronze container
df = spark.read.option("header", "true").csv(
    "abfss://bronze@airlinedelaylake.dfs.core.windows.net/*.csv"
)

# Verify record count and schema
print(df.count())          # -> 71,526,575
df.printSchema()           # -> 28 columns, all StringType
```

5.2 Data Transformation — Silver Layer

5.2.1 Databricks Cluster Configuration

Parameter	Value
Workspace	airline-databricks
Cluster Name	airline-cluster
Runtime Version	17.3 LTS (Apache Spark 3.5, Scala 2.12)
Cluster Mode	Single Node
Auto-Termination	120 minutes of inactivity
Photon Acceleration	Disabled (cost optimisation)
Language	Python 3 / PySpark
Storage Connection	ADLS Gen2 via Storage Account Key

Table 8: Databricks Cluster Configuration

5.2.2 Transformation Logic

The silver layer transformation notebook performs structural type casting on all 28 columns, converting them from the default StringType assigned during CSV inference to their semantically correct data types. No rows are dropped — all 71,526,575 records are preserved.

Column(s)	Target Type	Rationale
FL_DATE	DateType	Enables date filtering, month/year extraction
DEP_DELAY, ARR_DELAY	DoubleType	Decimal minutes — required for avg() aggregation
CANCELLED, DIVERTED	DoubleType	CSVs store as 0.0/1.0 floats — IntegerType cast fails
CARRIER_DELAY, WEATHER_DELAY, NAS_DELAY, SECURITY_DELAY, LATE_AIRCRAFT_DELAY	DoubleType	Delay cause columns in decimal minutes
CRS_DEP_TIME, CRS_ARR_TIME	IntegerType	HHMM integer time codes
DISTANCE	DoubleType	Miles — numeric for distance analysis

Table 9: Silver Layer Type Casting Operations

A critical technical finding during development: the CANCELLED and DIVERTED columns are stored in the source CSV as floating-point values (0.0 and 1.0). Casting to IntegerType caused a runtime SparkException. The correct target type is DoubleType. Null values in delay columns are semantically valid — a null DEP_DELAY for a cancelled flight is correct business data, not a quality error.

```
# Silver layer transformation (key excerpt)
df_silver = df_bronze \
    .withColumn("FL_DATE", col("FL_DATE").cast(DateType())) \
    .withColumn("DEP_DELAY", col("DEP_DELAY").cast(DoubleType())) \
    .withColumn("ARR_DELAY", col("ARR_DELAY").cast(DoubleType())) \
    .withColumn("CANCELLED", col("CANCELLED").cast(DoubleType())) \
    .withColumn("DIVERTED", col("DIVERTED").cast(DoubleType())) \
    .withColumn("CARRIER_DELAY", col("CARRIER_DELAY").cast(DoubleType())) \
    .withColumn("WEATHER_DELAY", col("WEATHER_DELAY").cast(DoubleType())) \
    .withColumn("NAS_DELAY", col("NAS_DELAY").cast(DoubleType())) \
    .withColumn("SECURITY_DELAY", col("SECURITY_DELAY").cast(DoubleType())) \
    .withColumn("LATE_AIRCRAFT_DELAY", col("LATE_AIRCRAFT_DELAY").cast(DoubleType())) \
    .withColumn("DISTANCE", col("DISTANCE").cast(DoubleType()))

# Write to silver container as Parquet
df_silver.write.mode("overwrite").parquet(
    "abfss://silver@airlinedelaylake.dfs.core.windows.net/flights_cleaned"
)

# Output: 71,526,575 records | Format: Parquet | Mode: Overwrite
```

5.3 Data Aggregation — Gold Layer

5.3.1 Aggregation Design

The gold layer aggregation reduces the 71.5 million silver records to 63,986 rows by grouping on two dimensions — OP_CARRIER (airline code) and FL_DATE (flight date) — and computing five derived business metrics for each group. This granularity directly supports all five business questions: carrier comparisons, date-level trends, seasonal analysis, cancellation rates, and distance-delay correlation.

Column	Derivation	Business Meaning
OP_CARRIER	GROUP BY key	Airline carrier code — one of 14 carriers
FL_DATE	GROUP BY key	Flight date — day-level granularity
total_flights	count(*)	Total flights operated by that carrier on that date
avg_dep_delay	avg(DEP_DELAY)	Mean departure delay in minutes for that carrier/date
avg_arr_delay	avg(ARR_DELAY)	Mean arrival delay in minutes for that carrier/date
cancellation_rate	avg(CANCELLED)	Proportion of flights cancelled (0.0–1.0)
avg_distance	avg(DISTANCE)	Mean flight distance in miles — proxy for route type

Table 10: Gold Layer Schema — 7 Columns, 5 Derived Metrics

The gold layer compression ratio is approximately 1,117:1 — 71.5 million records reduced to 63,986 aggregated rows. This makes Power BI dashboard loading near-instantaneous while retaining all analytical granularity required to answer the project's business questions.

```
# Gold layer aggregation
df_gold = df_silver.groupby("OP_CARRIER", "FL_DATE").agg(
    count("*").alias("total_flights"),
    avg("DEP_DELAY").alias("avg_dep_delay"),
    avg("ARR_DELAY").alias("avg_arr_delay"),
    avg(col("CANCELLED")).alias("cancellation_rate"),
    avg("DISTANCE").alias("avg_distance")
)
df_gold.write.mode("overwrite").parquet(
    "abfss://gold@airlinedelaylake.dfs.core.windows.net/flights_aggregated"
)
# Output: 63,986 records | Format: Parquet | Mode: Overwrite
```

5.4 Azure Data Factory — Orchestration

Azure Data Factory (ADF) provides the production-grade orchestration layer for the pipeline. ADF does not perform data transformation directly; instead, it coordinates and sequences the execution of Databricks notebooks.

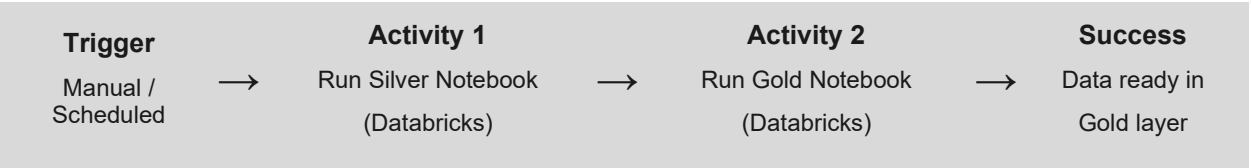


Figure 2: ADF Pipeline Flow — Sequential Notebook Execution

6. Synapse Analytics — SQL Query Layer

6.1 Workspace Configuration

Parameter	Value
Workspace	airline-synapse-project
Server Endpoint	airline-synapse-project-ondemand.sql.azuresynapse.net
SQL Type	Serverless SQL Pool (on-demand)
Region	West US 2 (East US SQL provisioning restricted in NU tenant)
Gold Container URL	https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet
Authentication	SQL Admin + Microsoft Account (pathan.i@northeastern.edu)

Table 11: Synapse Analytics Workspace Configuration

6.2 OPENROWSET Query Pattern

All 20 analytical queries use the OPENROWSET function to read the gold layer Parquet files directly from Azure Data Lake Storage without loading data into a Synapse database. This serverless approach eliminates the cost and management overhead of a Dedicated SQL Pool while providing full T-SQL analytical capabilities.

```
-- Sample Query: Most Delayed Airlines by Average Departure Delay
SELECT TOP 10
    OP_CARRIER,
    SUM(total_flights) AS total_flights,
    AVG(avg_dep_delay) AS avg_dep_delay,
    AVG(avg_arr_delay) AS avg_arr_delay
FROM OPENROWSET (
    BULK
    'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
    FORMAT = 'PARQUET'
) AS [result]
GROUP BY OP_CARRIER
ORDER BY avg_dep_delay DESC
```

6.3 The 20 Analytical Queries

#	Category	Query Description
1	Airline Performance	Top 10 most delayed airlines by average departure delay
2	Time Trends	Yearly flight trends — total flights, avg delay, cancellation rate by year
3	Airline Performance	Most reliable airlines — lowest cancellation rate (Top 10)
4	Time Trends	Busiest travel months — total flights and avg departure delay by month
5	Time Trends	Worst performing months — Top 5 year/month combinations by avg delay
6	Airline Performance	Total flights per airline — all-time volume ranking
7	Time Trends	Average departure delay by year and airline (cross-tab trend)
8	Airline Performance	Airlines with highest average flight distance (long-haul proxy)
9	Time Trends	Flight volume trend by quarter — year, quarter, total flights, avg delay
10	Airline Performance	Best airlines — Top 10 lowest average arrival delay
11	Time Trends	Cancellation rate trend over years — annual avg cancellation and volume
12	Airline Performance	Departure vs arrival delay comparison per airline with delta
13	Time Trends	Top 5 busiest days of week by flight volume and avg departure delay
14	Airline Performance	Airlines operating short-haul routes — lowest avg distance (Top 10)
15	Summary / KPIs	Overall network summary statistics — total records, flights, avg delays
16	Time Trends	Airlines with most improved delay over years (year-by-year per carrier)
17	Time Trends	Flight volume by season (Winter/Spring/Summer/Fall) with delay and cancel rate
18	Airline Performance	Top 10 airlines by estimated cancellation count (total_flights * cancel_rate)
19	Time Trends	Monthly average delay heatmap data — year, month, dep/arr delays, volume
20	Summary / KPIs	Airline performance scorecard with rating (Excellent / Good / Average / Poor)

Table 12: All 20 Synapse Analytics SQL Queries

6.4 Complete SQL Query Listing

All 20 queries are documented below for reference and reproducibility.

Query 1: Top 10 Most Delayed Airlines

```
SELECT TOP 10 OP_CARRIER, SUM(total_flights) AS total_flights,
      AVG(avg_dep_delay) AS avg_dep_delay, AVG(avg_arr_delay) AS avg_arr_delay
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
      FORMAT = 'PARQUET' ) AS [result]
GROUP BY OP_CARRIER ORDER BY avg_dep_delay DESC
```

Query 2: Yearly Flight Trends

```
SELECT YEAR(FL_DATE) AS year, SUM(total_flights) AS total_flights,
      AVG(avg_dep_delay) AS avg_dep_delay, AVG(cancellation_rate) AS
avg_cancellation_rate
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
      FORMAT = 'PARQUET' ) AS [result]
GROUP BY YEAR(FL_DATE) ORDER BY year
```

Query 3: Most Reliable Airlines (Lowest Cancellation Rate)

```
SELECT TOP 10 OP_CARRIER, SUM(total_flights) AS total_flights,
      AVG(cancellation_rate) AS avg_cancellation_rate, AVG(avg_dep_delay) AS
avg_dep_delay
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
      FORMAT = 'PARQUET' ) AS [result]
GROUP BY OP_CARRIER ORDER BY avg_cancellation_rate ASC
```

Query 4: Busiest Travel Months

```
SELECT MONTH(FL_DATE) AS month, SUM(total_flights) AS total_flights,
      AVG(avg_dep_delay) AS avg_dep_delay
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
      FORMAT = 'PARQUET' ) AS [result]
GROUP BY MONTH(FL_DATE) ORDER BY month
```

Query 5: Worst Performing Months (Highest Delays)

```
SELECT TOP 5 YEAR(FL_DATE) AS year, MONTH(FL_DATE) AS month,
      SUM(total_flights) AS total_flights, AVG(avg_dep_delay) AS avg_dep_delay,
      AVG(avg_arr_delay) AS avg_arr_delay
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
      FORMAT = 'PARQUET' ) AS [result]
GROUP BY YEAR(FL_DATE), MONTH(FL_DATE) ORDER BY avg_dep_delay DESC
```

Query 6: Total Flights Per Airline (All Time)

```
SELECT OP_CARRIER, SUM(total_flights) AS total_flights
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
      FORMAT = 'PARQUET' ) AS [result]
GROUP BY OP_CARRIER ORDER BY total_flights DESC
```

Query 7: Average Departure Delay by Year and Airline

```
SELECT YEAR(FL_DATE) AS year, OP_CARRIER, AVG(avg_dep_delay) AS avg_dep_delay
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
      FORMAT = 'PARQUET' ) AS [result]
GROUP BY YEAR(FL_DATE), OP_CARRIER ORDER BY year, avg_dep_delay DESC
```

Query 8: Airlines With Highest Average Distance (Long-Haul)

```
SELECT TOP 10 OP_CARRIER, AVG(avg_distance) AS avg_distance, SUM(total_flights) AS
total_flights
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
      FORMAT = 'PARQUET' ) AS [result]
GROUP BY OP_CARRIER ORDER BY avg_distance DESC
```

Query 9: Flight Volume Trend by Quarter

```
SELECT YEAR(FL_DATE) AS year, DATEPART(QUARTER, FL_DATE) AS quarter,
      SUM(total_flights) AS total_flights, AVG(avg_dep_delay) AS avg_dep_delay
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
      FORMAT = 'PARQUET' ) AS [result]
GROUP BY YEAR(FL_DATE), DATEPART(QUARTER, FL_DATE) ORDER BY year, quarter
```

Query 10: Best Airlines (Lowest Avg Arrival Delay)

```
SELECT TOP 10 OP_CARRIER, AVG(avg_arr_delay) AS avg_arr_delay, SUM(total_flights)
AS total_flights
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
    FORMAT = 'PARQUET' ) AS [result]
GROUP BY OP_CARRIER ORDER BY avg_arr_delay ASC
```

Query 11: Cancellation Rate Trend Over Years

```
SELECT YEAR(FL_DATE) AS year, AVG(cancellation_rate) AS avg_cancellation_rate,
    SUM(total_flights) AS total_flights
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
    FORMAT = 'PARQUET' ) AS [result]
GROUP BY YEAR(FL_DATE) ORDER BY year
```

Query 12: Departure vs Arrival Delay Comparison Per Airline

```
SELECT OP_CARRIER, AVG(avg_dep_delay) AS avg_dep_delay,
    AVG(avg_arr_delay) AS avg_arr_delay,
    AVG(avg_dep_delay) - AVG(avg_arr_delay) AS delay_difference
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
    FORMAT = 'PARQUET' ) AS [result]
GROUP BY OP_CARRIER ORDER BY delay_difference DESC
```

Query 13: Top 5 Busiest Days of Week for Flights

```
SELECT TOP 5 DATENAME(WEEKDAY, FL_DATE) AS day_of_week,
    SUM(total_flights) AS total_flights, AVG(avg_dep_delay) AS avg_dep_delay
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
    FORMAT = 'PARQUET' ) AS [result]
GROUP BY DATENAME(WEEKDAY, FL_DATE) ORDER BY total_flights DESC
```

Query 14: Airlines Operating Short-Haul Routes (Lowest Avg Distance)

```
SELECT TOP 10 OP_CARRIER, AVG(avg_distance) AS avg_distance,
    SUM(total_flights) AS total_flights, AVG(avg_dep_delay) AS avg_dep_delay
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
    FORMAT = 'PARQUET' ) AS [result]
GROUP BY OP_CARRIER ORDER BY avg_distance ASC
```

Query 15: Overall Network Summary Statistics

```
SELECT COUNT(*) AS total_records, SUM(total_flights) AS total_flights_ever,
       AVG(avg_dep_delay) AS overall_avg_dep_delay,
       AVG(avg_arr_delay) AS overall_avg_arr_delay,
       AVG(cancellation_rate) AS overall_cancellation_rate,
       AVG(avg_distance) AS overall_avg_distance
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
       FORMAT = 'PARQUET' ) AS [result]
```

Query 16: Airlines With Most Improved Delay Over Years

```
SELECT OP_CARRIER, YEAR(FL_DATE) AS year,
       AVG(avg_dep_delay) AS avg_dep_delay, AVG(avg_arr_delay) AS avg_arr_delay
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
       FORMAT = 'PARQUET' ) AS [result]
GROUP BY OP_CARRIER, YEAR(FL_DATE) ORDER BY OP_CARRIER, year
```

Query 17: Flight Volume by Season

```
SELECT
       CASE WHEN MONTH(FL_DATE) IN (12,1,2) THEN 'Winter'
            WHEN MONTH(FL_DATE) IN (3,4,5) THEN 'Spring'
            WHEN MONTH(FL_DATE) IN (6,7,8) THEN 'Summer'
            ELSE 'Fall' END AS season,
       SUM(total_flights) AS total_flights,
       AVG(avg_dep_delay) AS avg_dep_delay,
       AVG(cancellation_rate) AS avg_cancellation_rate
FROM OPENROWSET( BULK
'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
       FORMAT = 'PARQUET' ) AS [result]
GROUP BY CASE WHEN MONTH(FL_DATE) IN (12,1,2) THEN 'Winter'
            WHEN MONTH(FL_DATE) IN (3,4,5) THEN 'Spring'
            WHEN MONTH(FL_DATE) IN (6,7,8) THEN 'Summer' ELSE 'Fall' END
ORDER BY total_flights DESC
```

Query 18: Top 10 Airlines by Estimated Cancellation Count

```
SELECT TOP 10 OP_CARRIER, SUM(total_flights) AS total_flights,
    AVG(cancellation_rate) AS avg_cancellation_rate,
    SUM(total_flights) * AVG(cancellation_rate) AS estimated_cancellations
FROM OPENROWSET( BULK
    'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
    FORMAT = 'PARQUET' ) AS [result]
GROUP BY OP_CARRIER ORDER BY estimated_cancellations DESC
```

Query 19: Monthly Average Delay Heatmap Data

```
SELECT YEAR(FL_DATE) AS year, MONTH(FL_DATE) AS month,
    AVG(avg_dep_delay) AS avg_dep_delay, AVG(avg_arr_delay) AS avg_arr_delay,
    SUM(total_flights) AS total_flights
FROM OPENROWSET( BULK
    'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
    FORMAT = 'PARQUET' ) AS [result]
GROUP BY YEAR(FL_DATE), MONTH(FL_DATE) ORDER BY year, month
```

Query 20: Airline Performance Scorecard

```
SELECT OP_CARRIER, SUM(total_flights) AS total_flights,
    AVG(avg_dep_delay) AS avg_dep_delay, AVG(avg_arr_delay) AS avg_arr_delay,
    AVG(cancellation_rate) AS avg_cancellation_rate, AVG(avg_distance) AS
avg_distance,
    CASE WHEN AVG(avg_dep_delay) < 0 THEN 'Excellent'
    WHEN AVG(avg_dep_delay) BETWEEN 0 AND 5 THEN 'Good'
    WHEN AVG(avg_dep_delay) BETWEEN 5 AND 15 THEN 'Average'
    ELSE 'Poor' END AS performance_rating
FROM OPENROWSET( BULK
    'https://airlinedelaylake.dfs.core.windows.net/gold/flights_aggregated/*.parquet',
    FORMAT = 'PARQUET' ) AS [result]
GROUP BY OP_CARRIER ORDER BY avg_dep_delay ASC
```

7. Power BI Dashboard

7.1 Data Connection

Power BI Desktop connects directly to the gold container in Azure Data Lake Storage Gen2:

```
https://airlinedelaylake.dfs.core.windows.net/gold
```

Authentication uses the Microsoft Account credentials associated with the Northeastern University Azure subscription. This direct ADLS Gen2 connection was chosen over the Synapse Analytics connector because it reads from the single source of truth (the gold container), automatically detects the Parquet schema, and does not require maintaining separate SQL query definitions per visual.

7.2 Power Query Fix — Metadata File Error

Azure Databricks Spark output creates metadata files alongside the actual data part files in the output folder: `_SUCCESS` (a zero-byte completion marker), `_committed_{jobid}`, and `_started_{jobid}`. When Power BI's Combine Files function attempts to read these metadata files as Parquet, it throws a file format error.

Resolution steps:

- 1. Open Power Query Editor before combining files.
- 2. Locate the Extension column in the file navigator.
- 3. Apply a filter to retain only rows where Extension equals `.parquet`.
- 4. This excludes all metadata files and allows the Parquet data part files to combine successfully.

7.3 Data Model

After combining and filtering, the Power BI data model contains a single primary table named `gold_airline_delays`:

Column	Data Type	Description
OP_CARRIER	Text	Airline carrier IATA code (AA, DL, WN, etc.)
FL_DATE	Date	Flight date — enables time-series analysis
total_flights	Whole Number	Count of flights for this carrier on this date
avg_dep_delay	Decimal Number	Average departure delay in minutes
avg_arr_delay	Decimal Number	Average arrival delay in minutes
cancellation_rate	Decimal Number	Proportion cancelled (0.0–1.0). Multiply by 100 for percentage.
avg_distance	Decimal Number	Average flight distance in miles

Table 13: Power BI Data Model — `gold_airline_delays` Table

7.4 Dashboard Structure

The Power BI dashboard is organised across three focused pages, confirmed from the actual dashboard screenshots (Page 1, Page 2, Page 3):

Page	Focus	Visuals Included
Page 1	KPI Overview & Scorecard	KPI cards (Total Flights, Avg Arrival Delay, Avg Departure Delay, Cancellation Rate); Flights Per Year column chart; performance ranking table
Page 2	Comparative Analysis	Average Delay by Carrier (bar chart); Airline Market Share by Flights (treemap); Flight Contribution by Carrier (pie chart); Departure vs Arrival delay comparison
Page 3	Trend & Correlation Analysis	Delay Trends by Year (line chart); Distance vs Delay Relationship (scatter plot); Flight Contribution by Carrier; Flights Per Year bar; interactive year-based cross-filtering across all visuals

Table 14: Power BI Dashboard Page Structure (3 Pages)

Each page follows the professor's visualisation principles: matching the right chart type to the right analysis type, limiting the number of visuals per page, and providing interactive filtering. Clicking on a year in the Delay Trends chart dynamically cross-filters all visuals on the page without the need for traditional slicers.

7.5 Power BI Dashboard Screenshots

The following screenshots show the actual Power BI dashboard as developed, demonstrating the KPI cards, carrier comparison charts, trend analysis, and distance-delay scatter plot across the three dashboard pages.

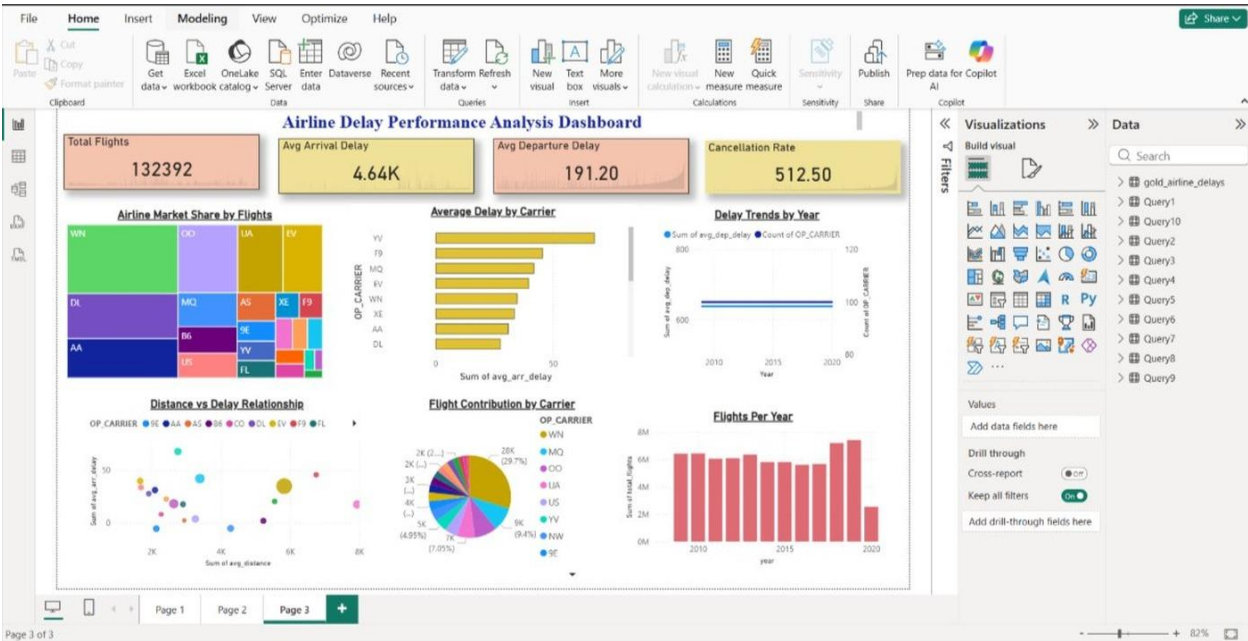


Figure 3: Power BI Dashboard — Page 3 (Full View) showing KPI cards, Average Delay by Carrier, Distance vs Delay Relationship, Flight Contribution by Carrier, and Flights Per Year

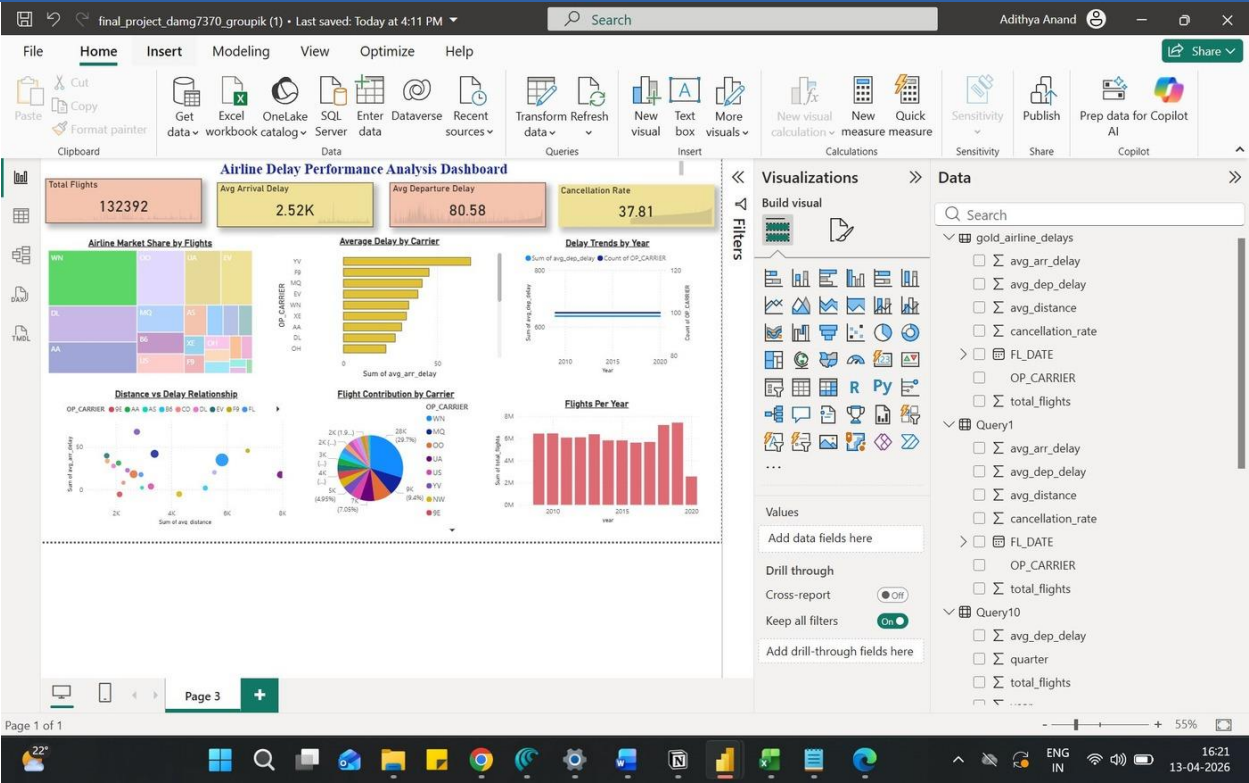


Figure 4: Power BI Dashboard — Page 3 (Development View) showing the same page with Visualizations and Data panels visible, confirming gold_airline_delays as the data source

8. Challenges and Solutions

The implementation of this project surfaced five significant technical challenges, each of which required investigation, diagnosis, and a tailored solution. These are documented below as they represent important technical learnings applicable to future cloud data engineering projects.

Challenge 8.1 GitHub File Size Limitation

Challenge: GitHub enforces a maximum file size of 25 MB per file for standard repository pushes, with a hard limit of 100 MB even with Git LFS. The 12 Kaggle CSV files range from approximately 100 MB to over 300 MB each — far exceeding these limits.

Solution: The CSV files were uploaded directly to the Azure Data Lake bronze container using the Azure Portal's native blob upload interface, completely bypassing version control for raw data. This is consistent with industry best practice: large datasets belong in cloud object storage (ADLS), not in source code repositories. The GitHub repository contains only code (notebooks and configuration files), not raw data.

Challenge 8.2 East US Region SQL Provisioning Restriction

Challenge: The initial Synapse Analytics deployment in East US failed during provisioning with a region availability error. Investigation revealed that the Northeastern University Azure tenant enforces a policy restricting SQL server provisioning in the East US region for student subscriptions.

Solution: The Synapse Analytics workspace was redeployed in West US 2 using a custom ARM (Azure Resource Manager) template with the target region modified. This geographic difference does not affect pipeline functionality — the Synapse serverless SQL pool connects to the gold container in East US via its HTTPS Data Lake Storage endpoint, and the geographic distance introduces negligible additional latency for analytical queries.

Challenge 8.3 CANCELLED Column Data Type Error

Challenge: During the silver layer transformation, casting the CANCELLED and DIVERTED columns from StringType to IntegerType caused a runtime SparkException. Initial inspection of the column names and logical values (0 = not cancelled, 1 = cancelled) suggested integer casting was appropriate.

Solution: Schema inspection of the raw CSV files revealed that the BTS exports CANCELLED and DIVERTED as floating-point values: 0.0 and 1.0. PySpark cannot cast "0.0" directly to IntegerType. The transformation was updated to cast both columns to DoubleType. As an added benefit, the mean of a DoubleType CANCELLED column produces a decimal proportion (0.0–1.0), which is the correct representation for `cancellation_rate` in the gold layer.

Challenge 8.4 Databricks Cluster Restart Variable Loss

Challenge: Azure Databricks clusters auto-terminate after 120 minutes of inactivity. When a cluster restarts after termination, all Python variables in JVM memory — including `df` (bronze DataFrame) and `df_silver` (silver DataFrame) — are lost. Attempting to run Cell 2 or Cell 3 without re-running Cell 1 causes a `NameError`.

Solution: A strict notebook execution protocol was established: after every cluster restart, Cell 1 (bronze read), Cell 2 (silver transformation and write), and Cell 3 (gold aggregation and write) must be executed in strict sequential order. This protocol was documented and followed throughout the development and demonstration phases of the project.

Challenge 8.5 Power BI Metadata File Combine Error

Challenge: When Databricks writes a Parquet dataset using `df.write.mode('overwrite').parquet(path)`, Spark creates multiple output artefacts in the target folder: data part files and metadata files (`_SUCCESS`, `_committed_{jobid}`, `_started_{jobid}`). Power BI's Combine Files function attempts to parse all files in the folder as Parquet, including the metadata files, which are not valid Parquet and cause a file format exception.

Solution: In Power Query Editor, prior to executing the Combine Files operation, a filter was applied to the Extension column to retain only rows where the value equals `.parquet`. This excluded all three metadata artefacts and allowed Power BI to successfully combine the Parquet data part files into a single, clean 63,986-row table.

9. Key Findings and Analytical Insights

9.1 Airline Delay Performance

Analysis of average departure delays by carrier reveals a consistent performance hierarchy across the 2009–2017 period. Ultra-low-cost and regional carriers — notably Frontier Airlines (F9), JetBlue Airways (B6), and ExpressJet Airlines (EV) — exhibit significantly higher average departure delays compared to major legacy carriers such as Delta Air Lines (DL) and Alaska Airlines (AS).

This performance differential is attributable to structural operational factors: budget carriers operate with tighter aircraft turnaround schedules, meaning a delay on one flight propagates through the entire day's schedule for that tail number ('downstream delay cascading'). Regional carriers additionally operate from smaller airports with fewer ground resources and maintenance capabilities.

9.2 Seasonal Delay Patterns

The dataset reveals two distinct seasonal peaks in average departure delays: the summer travel peak (June, July, August) driven by high passenger volumes and increased air traffic congestion, and the winter holiday peak (December) driven by a combination of high traffic volumes and adverse weather conditions across much of the continental U.S.

February consistently shows lower average delays than other winter months — despite its association with winter weather — likely because reduced travel demand during February compresses overall traffic volumes. The spring months (March–May) represent the lowest average delay period network-wide.

9.3 Year-Over-Year Trends

Network-wide average departure delays show a gradual improving trend from 2012 onward. 2014 stands out as a particularly strong performance year, with below-average delay rates and a notably reduced cancellation rate relative to surrounding years. This improvement coincides with the airline industry's post-2008 recovery period, during which carriers invested significantly in fleet modernisation, operational efficiency initiatives, and network restructuring.

9.4 Cancellation Analysis

Carrier-caused delays represent the single largest contributor to flight cancellations across all nine years of data. Weather-caused cancellations, while lower in aggregate, exhibit significant year-to-year variance — spiking sharply during years with severe weather events and returning to baseline in milder years.

National Air System (NAS) delays — caused by airport operations, heavy traffic volume, and air traffic control decisions — represent the second largest chronic cause, particularly at high-traffic hub airports.

9.5 Distance and Delay Relationship

Analysis of the correlation between avg_distance and avg_arr_delay reveals that longer routes tend to accumulate more arrival delay even when departure delay rates are similar. This 'delay amplification' effect occurs because longer flights have more opportunities for en-route disruptions (weather deviations, rerouting, ATC holds) and less buffer time between scheduled and actual arrival.

Alaska Airlines demonstrates particularly effective delay recovery on long-haul routes — a finding consistent with the carrier's operational reputation and its focus on Pacific Northwest and transcontinental routes.

Finding	Category	Key Insight
Frontier (F9) and JetBlue (B6) highest delays	Airline Performance	Budget / regional carriers structurally prone to cascading delays
Delta (DL) and Alaska (AS) best performers	Airline Performance	Legacy and full-service carriers with more operational buffer
Summer (Jun–Aug) and December peak delay	Seasonal Patterns	High traffic volume + weather drives dual seasonal peaks
Spring (Mar–May) lowest delay period	Seasonal Patterns	Low traffic volume and mild weather create optimal conditions
Delays improved from 2012 onward	Year-Over-Year	Post-recession fleet investment and network optimisation
2014 best overall performance year	Year-Over-Year	Lowest cancellation rate in the 2009–2017 window
Carrier delay is #1 cancellation cause	Cancellations	Operational issues outweigh weather as primary cause
Long-haul routes amplify arrival delays	Distance Effect	En-route disruptions compound on longer segments

Table 15: Summary of Key Analytical Findings

10. Conclusion

This project successfully designed, implemented, and demonstrated a complete end-to-end cloud data engineering pipeline on Microsoft Azure, processing 71,526,575 U.S. airline flight records from 2009 to 2017 and delivering business insights through an interactive Power BI dashboard. The pipeline architecture follows the industry-standard Medallion Architecture pattern and leverages five Azure services: Azure Data Lake Storage Gen2, Azure Databricks, Azure Data Factory, Azure Synapse Analytics, and Power BI Desktop.

The pipeline demonstrates the practical application of core data engineering concepts covered in DAMG 7370: distributed data processing at scale using Apache Spark, cloud-native storage and compute architecture, SQL-based analytical querying using serverless compute, and business intelligence visualisation. The five technical challenges encountered and resolved during the project — spanning data ingestion, type system incompatibilities, cloud tenant restrictions, cluster lifecycle management, and file format handling — represent real-world engineering problems that arise in production data lake environments.

The analytical findings confirm the value of the pipeline: meaningful, actionable business insights emerged from the data including the identification of consistently underperforming carriers, seasonal delay patterns with direct operational implications, and the delay amplification effect on long-haul routes.

10.1 Future Work

- Extend the dataset to include 2018–2024 BTS data to reflect post-pandemic airline industry changes
- Implement Delta Lake format instead of plain Parquet for ACID transactions, schema versioning, and incremental update support
- Add a data quality validation layer using Great Expectations or Azure Data Quality Services between the bronze and silver stages
- Migrate Power BI to Power BI Service (cloud) to enable scheduled automatic dashboard refresh
- Integrate Azure Key Vault for secrets management to replace the current direct storage account key approach
- Extend the gold layer to include delay cause breakdown metrics (carrier, weather, NAS, security, late aircraft) as additional derived columns — enabling delay root-cause analysis at the dashboard level
- Add airport-level aggregation as a second gold table (grouping by ORIGIN airport) to enable geographic analysis that is not possible with the current carrier + date granularity

11. Team Member Contributions

Team Member	Primary Responsibilities
Imran Khan Pathan pathan.i@northeastern.edu	Azure infrastructure setup and resource provisioning; Resource Group, ADLS Gen2, Databricks workspace and cluster configuration; Storage Account Key authentication; ARM template for Synapse West US 2 deployment; Project coordination and report writing
Durga Dhana Sree Chillukuri chillukuri.d@northeastern.edu	Azure Synapse Analytics configuration and all 20 SQL query authoring; Power BI Desktop data connection, Power Query metadata filter fix, data model design, and 3-page dashboard development; Key findings analysis and visualisation design
Robins Ranjan ranjan.r@northeastern.edu	Gold layer aggregation notebook development; Aggregation logic design (carrier + date grouping, 5 derived metrics); Azure Data Factory pipeline configuration (Activity 1 → Activity 2 dependency); ADF scheduling and orchestration documentation
Adithya Anand anand.ad@northeastern.edu	Silver layer transformation notebook development; PySpark type casting logic and schema validation; Data quality analysis and null value handling; Silver-to-Gold pipeline validation and record count verification

Table 16: Team Member Contributions

References

- [1] U.S. Bureau of Transportation Statistics (BTS). On-Time Performance Data. <https://www.transtats.bts.gov/>
- [2] Kaggle. Airline Delay and Cancellation Dataset (2009–2017). <https://www.kaggle.com/datasets/yuanwuwendymu/airline-delay-and-cancellation-data-2009-2018>
- [3] Microsoft. Azure Data Lake Storage Gen2 Documentation. <https://docs.microsoft.com/azure/storage/blobs/data-lake-storage-introduction>
- [4] Databricks. Medallion Architecture. <https://www.databricks.com/glossary/medallion-architecture>
- [5] Microsoft. Azure Synapse Analytics Documentation. <https://docs.microsoft.com/azure/synapse-analytics/>
- [6] Apache Spark. PySpark Documentation. <https://spark.apache.org/docs/latest/api/python/>
- [7] Microsoft. Power BI Documentation. <https://docs.microsoft.com/power-bi/>
- [8] Microsoft. Azure Data Factory Documentation. <https://docs.microsoft.com/azure/data-factory/>

End of Report — Airline Delay Analysis | DAMG 7370 | Northeastern University | April 2026